

Dr. RVR NRI Institute of Technology
Deemed to be University

PYTHON PROGRAMMING

Problem Solving Programs 11 – 20

Problem Statement • Objective • Assumptions • Algorithm • Flowchart
Pseudocode • Python Source Code • Test Cases • Output • Conclusion

Name	:	N Leela Saraswathi
Roll Number	:	26000047
Section	:	CSE 1

Index

Program 11: Examination Result and Grace-Mark Validation 3

Program 12: Attendance Recovery Computation 7

Program 13: ATM Transaction Sequence Simulation 11

Program 14: Warehouse Inventory Reconciliation 15

Program 15: Smart Parking Fee Management 19

Program 16: Multi-Day Water Tank Simulation 23

Program 17: Student Performance Trend Analysis 27

Program 18: Fuel Consumption and Route Comparison 31

Program 19: Multi-Product Shopping Budget Constraint 35

Program 20: University Admission Decision Engine 39

Each program contains: 1 Problem Statement, 2 Objective, 3 Assumptions, 4 Algorithm, 5 Flowchart, 6 Pseudocode, 7 Python Source Code, 8 Test Cases, 9 Output, 10 Conclusion.

Program 11: Examination Result and Grace-Mark Validation

1. Problem Statement

Process marks across multiple subjects, apply subject-wise pass criteria and a specified grace-mark policy, and generate final classifications without exceeding maximum marks.

2. Objective

To read the marks of every subject, validate them, apply the grace-mark rule to marks that fall just below the pass mark, and classify the final result as FAIL, PASS, SECOND CLASS, FIRST CLASS or DISTINCTION.

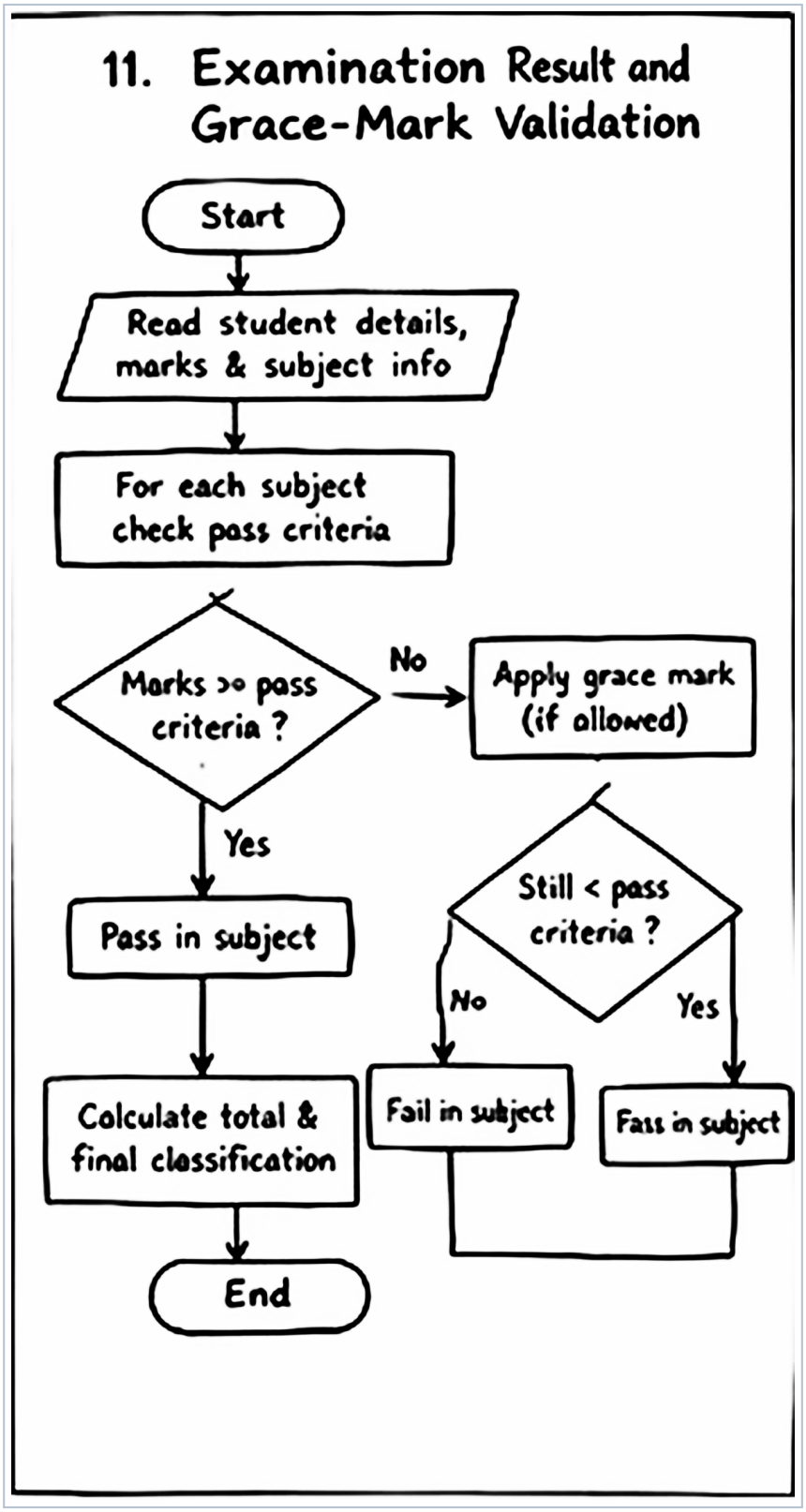
3. Assumptions

- Marks are whole numbers from 0 to 100. Any other value stops the program with the message "Invalid marks".
- The pass mark in every subject is 35.
- A grace of 5 marks is given only to a subject that scores from 30 to 34, and a mark never exceeds 100.
- Classification uses the percentage: 75 or more is DISTINCTION, 60 or more is FIRST CLASS, 50 or more is SECOND CLASS, otherwise PASS. Failing any subject gives FAIL.

4. Algorithm

1. Start and read the number of subjects n.
2. Create an empty list marks.
3. For each subject, read the mark m. If m < 0 or m > 100, display "Invalid marks" and stop.
4. If 30 ≤ m < 35, set m = min(m + 5, 100).
5. Add m to the list marks.
6. Compute total = sum of marks and percentage = total / n.
7. Display the final marks, the total and the percentage.
8. If any mark is below 35, display FAIL; otherwise classify by percentage (≥ 75, ≥ 60, ≥ 50, else PASS).
9. Stop.

5. Flowchart



Flowchart – Program 11: Examination Result and Grace-Mark Validation

6. Pseudocode

```
START
  READ n
  marks ← empty list
  FOR i ← 1 TO n DO
    READ m
    IF m < 0 OR m > 100 THEN
      DISPLAY "Invalid marks"
      STOP
    ENDIF
    IF m ≥ 30 AND m < 35 THEN
      m ← MIN(m + 5, 100)
    ENDIF
    ADD m TO marks
  ENDFOR
  total ← SUM(marks)
  percentage ← total / n
  DISPLAY marks, total, percentage
  IF any mark in marks < 35 THEN
    DISPLAY "FAIL"
  ELSE IF percentage ≥ 75 THEN
    DISPLAY "DISTINCTION"
  ELSE IF percentage ≥ 60 THEN
    DISPLAY "FIRST CLASS"
  ELSE IF percentage ≥ 50 THEN
    DISPLAY "SECOND CLASS"
  ELSE
    DISPLAY "PASS"
  ENDIF
END
```

7. Python Source Code

```
n = int(input("Enter number of subjects: "))
marks = []

for i in range(n):
    m = int(input(f"Enter marks for subject {i+1}: "))
    if m < 0 or m > 100:
        print("Invalid marks")
        exit()
    if 30 <= m < 35:
        m = min(m + 5, 100)
    marks.append(m)

total = sum(marks)
percentage = total / n

print("Final Marks:", marks)
print("Total:", total)
print("Percentage:", round(percentage, 2), "%")

if any(m < 35 for m in marks):
    print("Result: FAIL")
elif percentage >= 75:
    print("Result: DISTINCTION")
elif percentage >= 60:
    print("Result: FIRST CLASS")
elif percentage >= 50:
    print("Result: SECOND CLASS")
else:
    print("Result: PASS")
```

8. Test Cases

No.	Input	Expected Output	Result

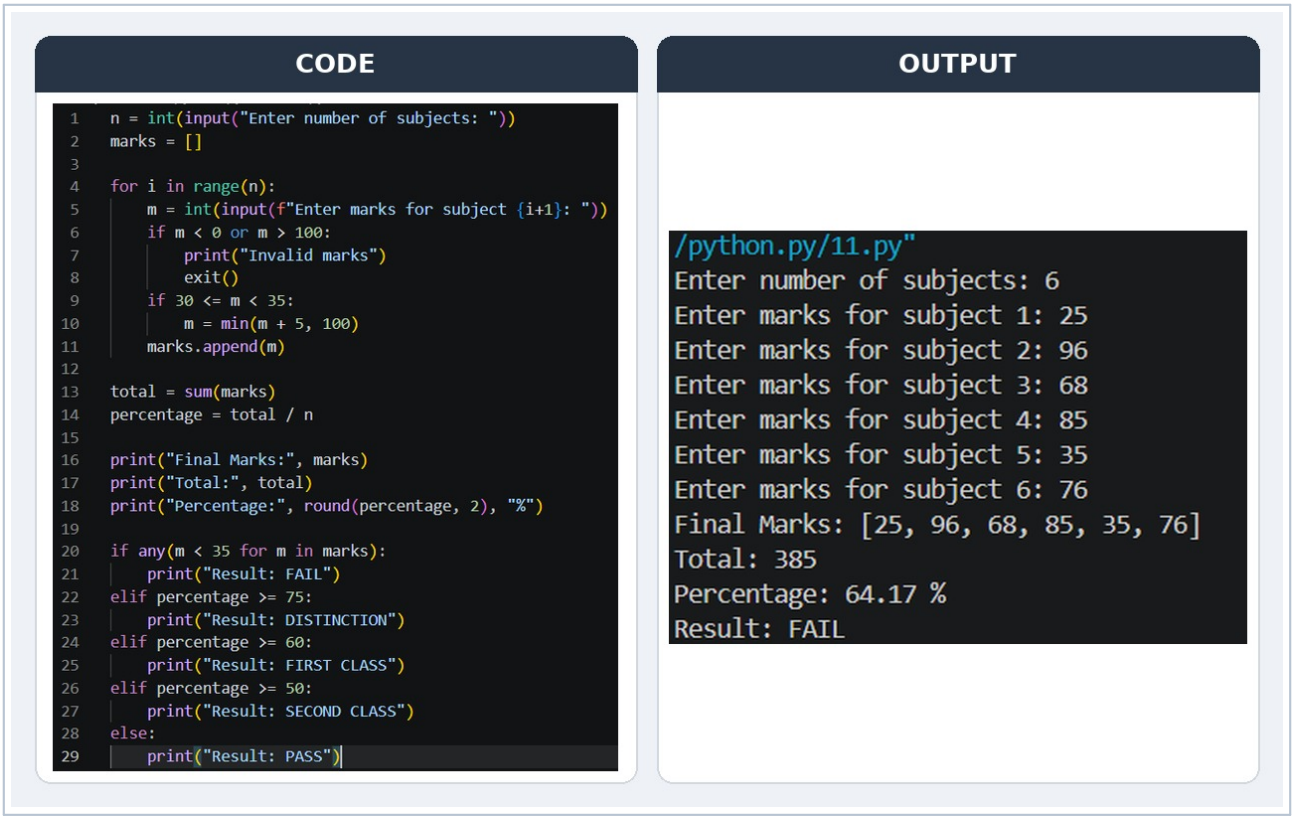
1	3 subjects: 80, 70, 90
	`Inputs: 3, 80, 70, 90`
		`Final Marks: [80, 70, 90]`
		`Total: 240`
		`Percentage: 80.0 %`
		`Result: DISTINCTION`
			Pass

2	3 subjects: 32, 60, 55 (grace mark on 32)
	`Inputs: 3, 32, 60, 55`
		`Final Marks: [37, 60, 55]`
		`Total: 152`
		`Percentage: 50.67 %`
		`Result: SECOND CLASS`
			Pass

3	2 subjects: 28, 90 (below grace range)
	`Inputs: 2, 28, 90`
		`Final Marks: [28, 90]`
		`Total: 118`
		`Percentage: 59.0 %`
		`Result: FAIL`
			Pass

4	2 subjects: 105, 50 (invalid marks) `Inputs: 2, 105, 50`	`Invalid marks`	**Pass**

9. Output



Output screenshot – Program 11 (code and output)

10. Conclusion

The program validates every mark, applies the grace-mark rule without crossing 100, and classifies the result correctly. The test cases for distinction, grace marks, failure and invalid input all gave the expected results.

Program 12: Attendance Recovery Computation

1. Problem Statement

Calculate current attendance percentages and determine the minimum consecutive classes required to reach an eligibility threshold under different future attendance scenarios.

2. Objective

To calculate the current attendance percentage and, if the student is below the eligibility percentage, find the minimum number of consecutive classes that must be attended to become eligible.

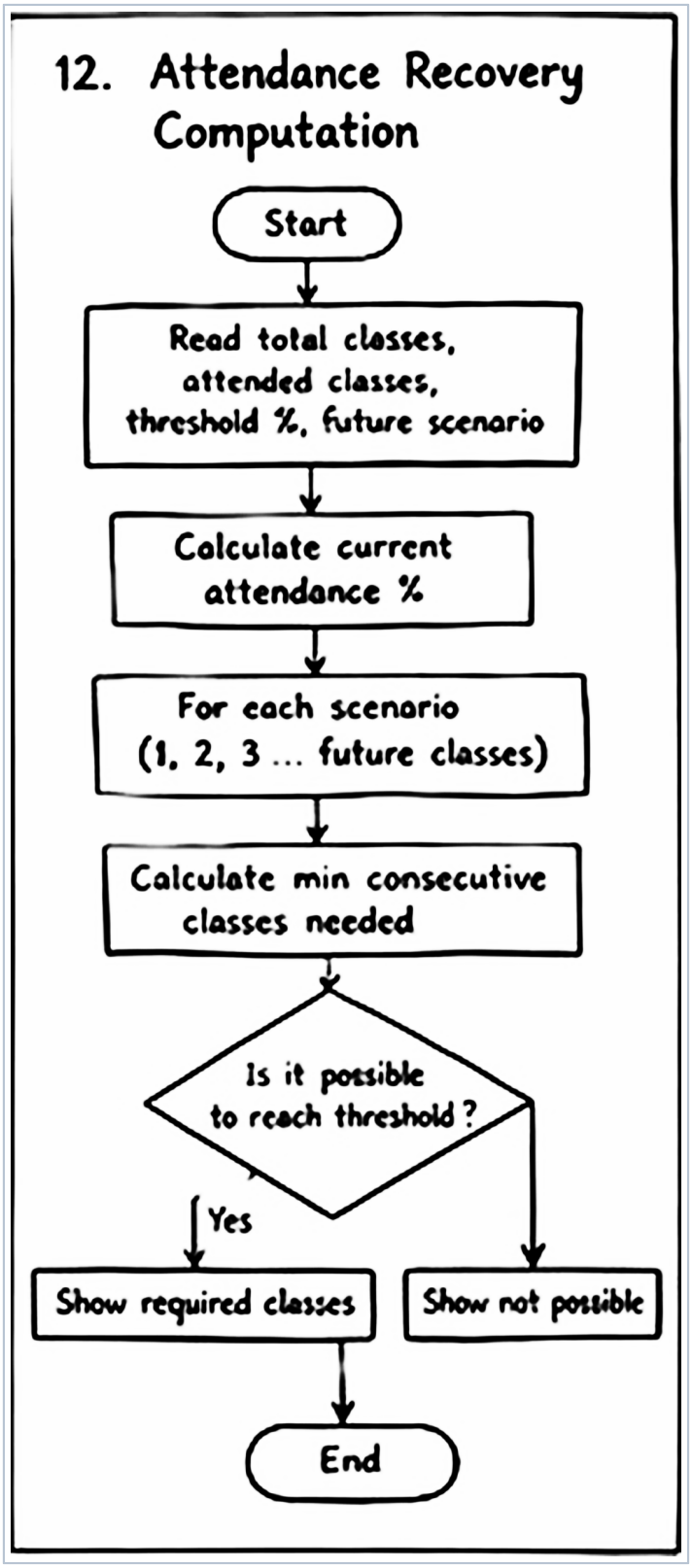
3. Assumptions

- Classes attended is not more than total classes, and total classes is greater than 0.
- The eligibility percentage is below 100. If it is 100 and the student has any absence, the target can never be reached.
- The future scenario considered is that the student attends every upcoming class, so both attended and total classes increase by 1 for each class.
- Other scenarios are tested by running the program again with different inputs.

4. Algorithm

1. Start. Read classes attended, total classes and the eligibility percentage.
2. Compute percentage = attended / total × 100 and display it.
3. If percentage ≥ threshold, display "Already Eligible" and stop.
4. Otherwise set needed = 0.
5. While (attended + needed) / (total + needed) × 100 < threshold, increase needed by 1.
6. Display needed and the attendance percentage after recovery.
7. Stop.

5. Flowchart



Flowchart – Program 12: Attendance Recovery Computation

6. Pseudocode

```
START
  READ attended, total, threshold
  percentage ← attended / total × 100
  DISPLAY percentage
  IF percentage ≥ threshold THEN
    DISPLAY "Already Eligible"
  ELSE
    needed ← 0
    WHILE (attended + needed) / (total + needed) × 100 < threshold DO
      needed ← needed + 1
    ENDWHILE
    DISPLAY needed
    DISPLAY (attended + needed) / (total + needed) × 100
  ENDIF
END
```

7. Python Source Code

```
attended = int(input("Enter classes attended: "))
total = int(input("Enter total classes: "))
threshold = float(input("Enter eligibility percentage: "))

percentage = attended / total * 100
print("Current Attendance:", round(percentage, 2), "%")

if percentage >= threshold:
    print("Already Eligible")
else:
    needed = 0
    while (attended + needed) / (total + needed) * 100 < threshold:
        needed += 1
    print("Consecutive classes required:", needed)
    print("Attendance after recovery:",
          round((attended + needed) / (total + needed) * 100, 2), "%")
```

8. Test Cases

No.	Input	Expected Output	Result
1	Attended 40, total 50, eligibility 75 `Inputs: 40, 50, 75`	`Current Attendance: 80.0 %` `Already Eligible`	**Pass**

2	Attended 30, total 50, eligibility 75
	`Inputs: 30, 50, 75`
		`Current Attendance: 60.0 %`
		`Consecutive classes required: 30`
		`Attendance after recovery: 75.0 %`
			Pass

3	Attended 60, total 80, eligibility 90
	`Inputs: 60, 80, 90`
		`Current Attendance: 75.0 %`
		`Consecutive classes required: 120`
		`Attendance after recovery: 90.0 %`
			Pass

9. Output



Output screenshot – Program 12 (code and output)

10. Conclusion

The program correctly reports the current attendance and finds the smallest number of consecutive classes needed to reach the eligibility percentage. Test cases confirmed both the eligible and the recovery paths.

Program 13: ATM Transaction Sequence Simulation

1. Problem Statement

Process a sequence of deposits and withdrawals, maintain a simulated balance, enforce withdrawal limits, reject invalid transactions, and produce a transaction summary.

2. Objective

To simulate an ATM that processes a list of deposits and withdrawals, keeps the balance updated, rejects invalid or over-limit requests, and finally prints the successful transactions and the final balance.

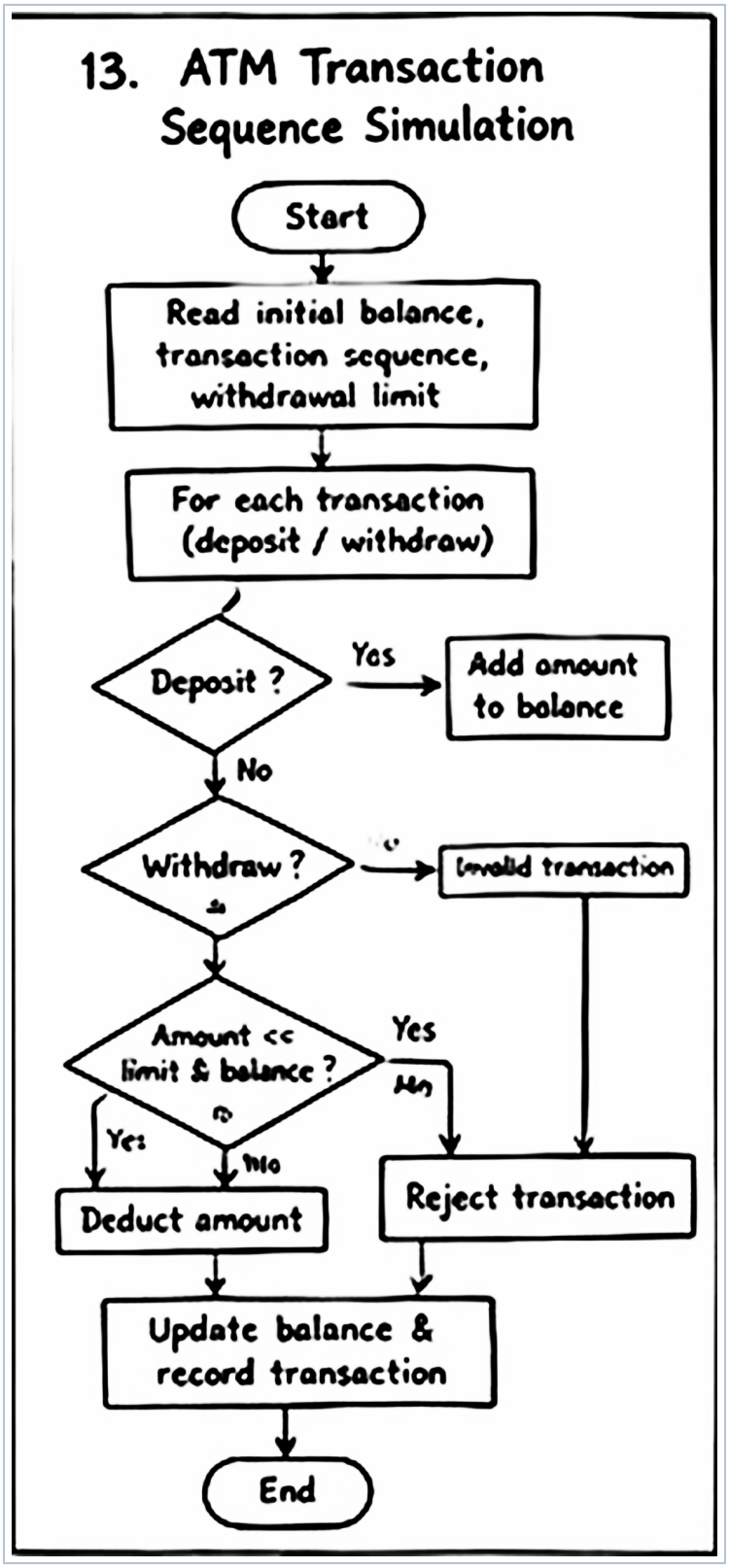
3. Assumptions

- The withdrawal limit is 5000 per transaction.
- The transaction type is typed as deposit or withdraw (upper or lower case is accepted). Any other type is an invalid transaction.
- The amount must be greater than 0, otherwise it is an invalid amount.
- A withdrawal is allowed only when the amount is not more than the balance and not more than the limit.
- Rejected or invalid transactions are not added to the summary. Amounts are decimal numbers, so very small floating-point differences can appear in the printed balance (for example 999.5999999999999).

4. Algorithm

1. Start. Read the initial balance, set limit = 5000 and create an empty list transactions.
2. Read the number of transactions n.
3. For each transaction, read the type (converted to lower case) and the amount.
4. If amount ≤ 0, display "Invalid amount".
5. Else if type is deposit, add the amount to the balance and record the deposit.
6. Else if type is withdraw: if amount ≤ balance and amount ≤ limit, subtract it and record the withdrawal; otherwise display "Withdrawal rejected".
7. Else display "Invalid transaction".
8. After all transactions, display the list of transactions and the final balance.
9. Stop.

5. Flowchart



Flowchart – Program 13: ATM Transaction Sequence Simulation

6. Pseudocode

```
START
  READ balance
  limit ← 5000
  transactions ← empty list
  READ n
  FOR i ← 1 TO n DO
    READ type, amount
    IF amount ≤ 0 THEN
      DISPLAY "Invalid amount"
    ELSE IF type = "deposit" THEN
      balance ← balance + amount
      ADD "Deposit: amount" TO transactions
    ELSE IF type = "withdraw" THEN
      IF amount ≤ balance AND amount ≤ limit THEN
        balance ← balance - amount
        ADD "Withdraw: amount" TO transactions
      ELSE
        DISPLAY "Withdrawal rejected"
      ENDIF
    ELSE
      DISPLAY "Invalid transaction"
    ENDIF
  ENDFOR
  DISPLAY transactions
  DISPLAY balance
END
```

7. Python Source Code

```
balance = float(input("Enter initial balance: "))
limit = 5000
transactions = []

n = int(input("Enter number of transactions: "))

for i in range(n):
    typ = input("Enter deposit/withdraw: ").lower()
    amt = float(input("Enter amount: "))

    if amt <= 0:
        print("Invalid amount")
    elif typ == "deposit":
        balance += amt
        transactions.append("Deposit: " + str(amt))
    elif typ == "withdraw":
        if amt <= balance and amt <= limit:
            balance -= amt
            transactions.append("Withdraw: " + str(amt))
        else:
            print("Withdrawal rejected")
    else:
        print("Invalid transaction")

print("\nTransactions:", transactions)
print("Final balance:", balance)
```

8. Test Cases

No.	Input	Expected Output	Result
1	Balance 1000; deposit 500, withdraw 300 `Inputs: 1000, 2, deposit, 500, withdraw, 300`	`Transactions: ['Deposit: 500.0', 'Withdraw: 300.0']` `Final balance: 1200.0`	**Pass**

2	Balance 2000; withdraw 6000 (over limit)
	`Inputs: 2000, 1, withdraw, 6000`
		`Withdrawal rejected`
		
		`Transactions: []`
		`Final balance: 2000.0`
			Pass

3	Balance 3000; withdraw 100, transfer 50
	`Inputs: 3000, 2, withdraw, 100, transfer, 50`
		`Invalid transaction`
		
		`Transactions: ['Withdraw: 100.0']`
		`Final balance: 2900.0`
			Pass

4	Balance 500; deposit -20
	`Inputs: 500, 1, deposit, -20`
		`Invalid amount`
		
		`Transactions: []`
		`Final balance: 500.0`
			Pass

9. Output

CODE

```python
balance = float(input("Enter initial balance: "))
limit = 5000
transactions = []

n = int(input("Enter number of transactions: "))

for i in range(n):
    typ = input("Enter deposit/withdraw: ").lower()
    amt = float(input("Enter amount: "))

    if amt <= 0:
        print("Invalid amount")
    elif typ == "deposit":
        balance += amt
        transactions.append("Deposit: " + str(amt))
    elif typ == "withdraw":
        if amt <= balance and amt <= limit:
            balance -= amt
            transactions.append("Withdraw: " + str(amt))
        else:
            print("Withdrawal rejected")
    else:
        print("Invalid transaction")

print("\nTransactions:", transactions)
print("Final balance:", balance)
```

OUTPUT

```
/python.py/13.py"
Enter initial balance: 5000.2
Enter number of transactions: 2
Enter deposit/withdraw: withdraw
Enter amount: 4000.60
Enter deposit/withdraw: withdraw
Enter amount: 1000.7
Withdrawal rejected

Transactions: ['Withdraw: 4000.6']
Final balance: 999.5999999999999
```

Output screenshot – Program 13 (code and output)

10. Conclusion

The simulation keeps the balance correct, enforces the withdrawal limit, rejects invalid requests and prints a clear summary. All test cases, including over-limit and invalid input, behaved as expected.

Program 14: Warehouse Inventory Reconciliation

1. Problem Statement

Compare opening stock, incoming quantities, dispatched quantities, and closing stock across multiple products; detect discrepancies and calculate replenishment requirements.

2. Objective

To compare the expected closing stock of each product with the actual closing stock, report any discrepancy, and calculate how many units must be ordered to restore the stock level.

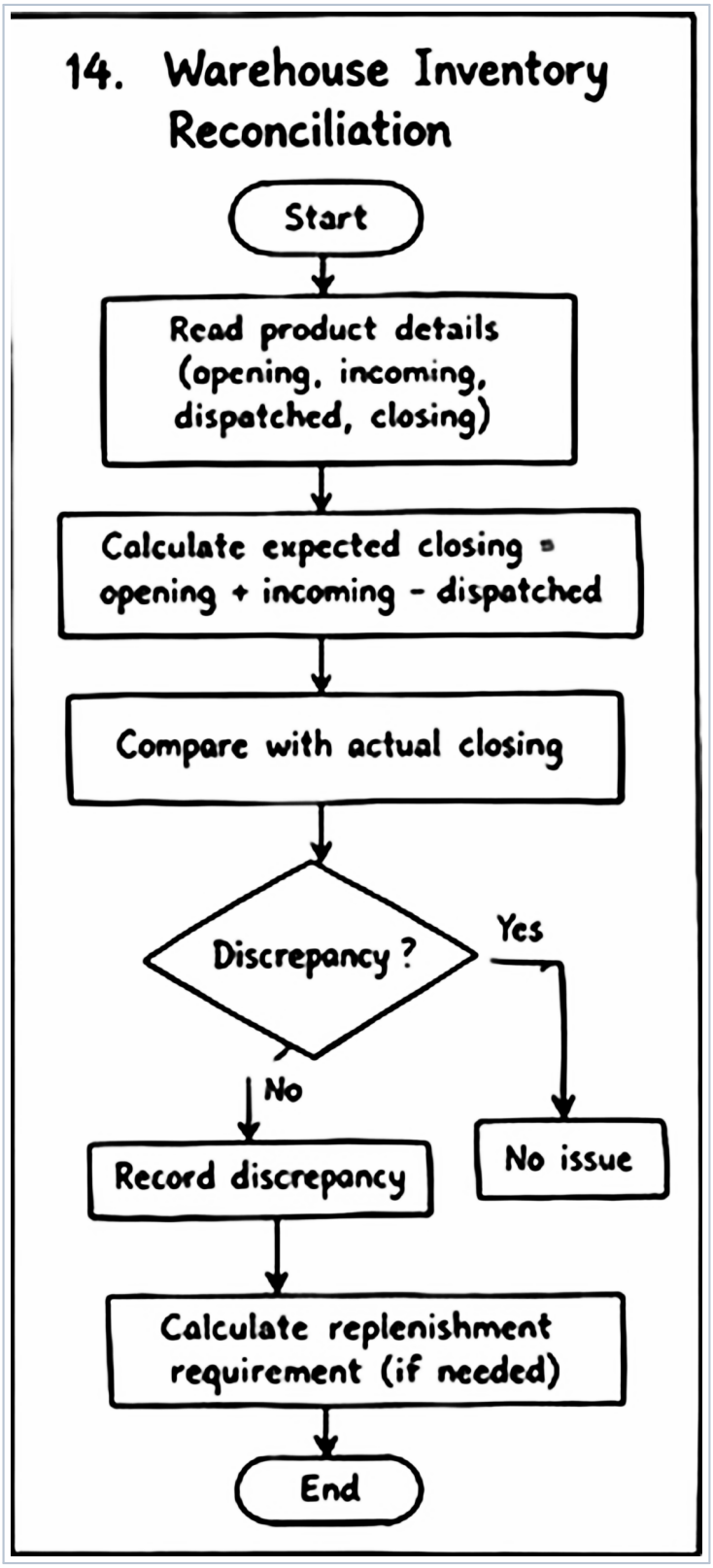
3. Assumptions

- All stock quantities are whole numbers that are not negative.
- Expected closing stock = opening stock + incoming stock – dispatched stock.
- Discrepancy = actual closing stock – expected closing stock. A negative value means missing stock and a positive value means extra stock.
- The target stock level of every product is 50 units, so replenishment = max(0, 50 – actual closing stock).

4. Algorithm

1. Start. Read the number of products n.
2. For each product, read the name, opening, incoming, dispatched and actual closing stock.
3. Compute expected = opening + incoming – dispatched and display it.
4. If closing = expected, display "Stock matches"; otherwise display the discrepancy (closing – expected).
5. Compute reorder = max(0, 50 – closing) and display it.
6. Repeat for all products, then stop.

5. Flowchart



Flowchart – Program 14: Warehouse Inventory Reconciliation

6. Pseudocode

```
START
  READ n
  FOR i ← 1 TO n DO
    READ name, opening, incoming, dispatched, closing
    expected ← opening + incoming - dispatched
    DISPLAY name, expected
    IF closing = expected THEN
      DISPLAY "Stock matches"
    ELSE
      DISPLAY "Discrepancy:", closing - expected
    ENDIF
    reorder ← MAX(0, 50 - closing)
    DISPLAY "Replenishment required:", reorder
  ENDFOR
END
```

7. Python Source Code

```
n = int(input("Enter number of products: "))

for i in range(n):
    name = input("Product name: ")
    opening = int(input("Opening stock: "))
    incoming = int(input("Incoming stock: "))
    dispatched = int(input("Dispatched stock: "))
    closing = int(input("Actual closing stock: "))

    expected = opening + incoming - dispatched
    print("Product:", name)
    print("Expected stock:", expected)

    if closing == expected:
        print("Stock matches")
    else:
        print("Discrepancy:", closing - expected)

    reorder = max(0, 50 - closing)
    print("Replenishment required:", reorder)
```

8. Test Cases

No.	Input	Expected Output	Result

1	1 product: pen, 20, 10, 5, closing 25
	`Inputs: 1, pen, 20, 10, 5, 25`
		`Product: pen`
		`Expected stock: 25`
		`Stock matches`
		`Replenishment required: 25`
			Pass

2	1 product: soap, 10, 5, 8, closing 4
	`Inputs: 1, soap, 10, 5, 8, 4`
		`Product: soap`
		`Expected stock: 7`
		`Discrepancy: -3`
		`Replenishment required: 46`
			Pass

3	2 products: rice (100,0,20,80) and oil (30,10,15,30)
	`Inputs: 2, rice, 100, 0, 20, 80, oil, 30, 10, 15, 30`
		`Product: rice`
		`Expected stock: 80`
		`Stock matches`
		`Replenishment required: 0`
		`Product: oil`
		`Expected stock: 25`
		`Discrepancy: 5`
		`Replenishment required: 20`
			Pass

9. Output

CODE

```python
n = int(input("Enter number of products: "))

for i in range(n):
    name = input("Product name: ")
    opening = int(input("Opening stock: "))
    incoming = int(input("Incoming stock: "))
    dispatched = int(input("Dispatched stock: "))
    closing = int(input("Actual closing stock: "))

    expected = opening + incoming - dispatched
    print("Product:", name)
    print("Expected stock:", expected)

    if closing == expected:
        print("Stock matches")
    else:
        print("Discrepancy:", closing - expected)

    reorder = max(0, 50 - closing)
    print("Replenishment required:", reorder)
```

OUTPUT

```
/python.py/14.py"
Enter number of products: 2
Product name: cusmotics
Opening stock: 8
Incoming stock: 7
Dispatched stock: 5
Actual closing stock: 2
Product: cusmotics
Expected stock: 10
Discrepancy: -8
Replenishment required: 48
Product name: hair oils
Opening stock: 8
Incoming stock: 7
Dispatched stock: 9
Actual closing stock: 5
Product: hair oils
Expected stock: 6
Discrepancy: -1
Replenishment required: 45
```

Output screenshot – Program 14 (code and output)

10. Conclusion

The program reconciles every product, detects shortages and excess stock, and calculates the replenishment quantity. The test cases with matching stock, shortage and excess stock all produced the expected values.

Program 15: Smart Parking Fee Management

1. Problem Statement

Calculate charges from entry and exit times, vehicle categories, overnight parking, daily limits, and penalty conditions while validating time inputs.

2. Objective

To validate the entry and exit times, calculate the parking duration, and compute the parking fee for the chosen vehicle category using an hourly rate and a maximum fee limit.

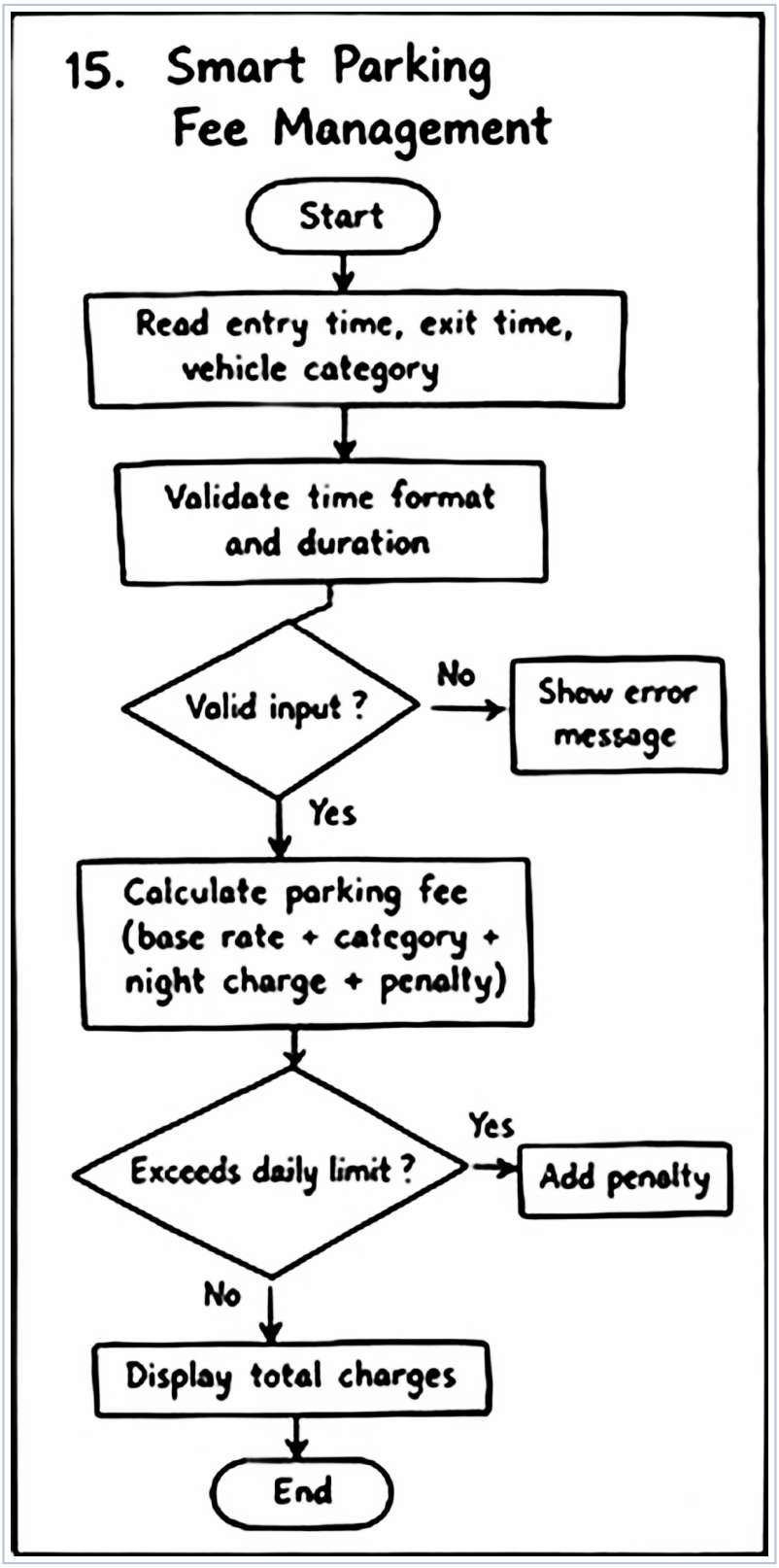
3. Assumptions

- Time is entered in the format YYYY-MM-DD HH:MM (24-hour clock). Any other format gives "Invalid time format".
- The exit time must be later than the entry time, otherwise the program shows "Invalid time".
- Hourly rates: car 30, bike 10, bus 50. The vehicle type is accepted in upper or lower case; any other type is "Invalid vehicle".
- A part of an hour is charged as a full hour (the hours value is rounded up).
- The fee is limited to a maximum of 500 and this limit is applied to the complete stay.
- Separate night charges and penalty amounts are not applied in this version of the program.

4. Algorithm

1. Start. Read the entry time and the exit time as text.
2. Try to convert both to date-time values. If the conversion fails, display "Invalid time format" and stop.
3. If exit ≤ entry, display "Invalid time" and stop.
4. Compute hours = (exit – entry) in seconds / 3600.
5. Read the vehicle type. If it is not car, bike or bus, display "Invalid vehicle" and stop.
6. Compute fee = min(500, rounded-up hours × hourly rate of the vehicle).
7. Display the parking fee.
8. Stop.

5. Flowchart



Flowchart – Program 15: Smart Parking Fee Management

6. Pseudocode

```
START
  READ entry, exit
  TRY
    e ← CONVERT entry TO date-time
    x ← CONVERT exit TO date-time
    IF x ≤ e THEN
      DISPLAY "Invalid time"
    ELSE
      hours ← (x - e) IN SECONDS / 3600
      rate ← {car: 30, bike: 10, bus: 50}
      READ vehicle
      IF vehicle NOT IN rate THEN
        DISPLAY "Invalid vehicle"
      ELSE
        fee ← MIN(500, ROUNDUP(hours) × rate[vehicle])
        DISPLAY fee
      ENDIF
    ENDIF
  ON ERROR
    DISPLAY "Invalid time format"
END
```

7. Python Source Code

```
from datetime import datetime

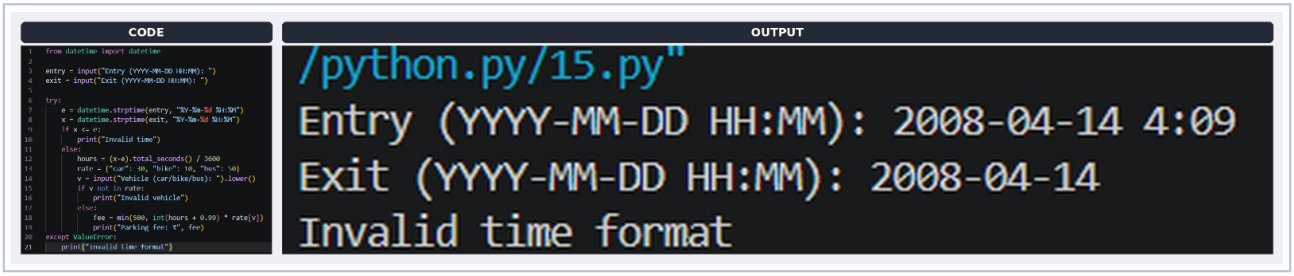
entry = input("Entry (YYYY-MM-DD HH:MM): ")
exit = input("Exit (YYYY-MM-DD HH:MM): ")

try:
    e = datetime.strptime(entry, "%Y-%m-%d %H:%M")
    x = datetime.strptime(exit, "%Y-%m-%d %H:%M")
    if x <= e:
        print("Invalid time")
    else:
        hours = (x - e).total_seconds() / 3600
        rate = {"car": 30, "bike": 10, "bus": 50}
        v = input("Vehicle (car/bike/bus): ").lower()
        if v not in rate:
            print("Invalid vehicle")
        else:
            fee = min(500, int(hours + 0.99) * rate[v])
            print("Parking fee: ₹", fee)
except ValueError:
    print("Invalid time format")
```

8. Test Cases

No.	Input	Expected Output	Result
1	Entry 2024-05-10 09:00, exit 2024-05-10 11:30, car `Inputs: 2024-05-10 09:00, 2024-05-10 11:30, car`	`Parking fee: ₹ 90`	**Pass**
2	Entry 2024-05-10 09:00, exit 2024-05-12 09:00, bus `Inputs: 2024-05-10 09:00, 2024-05-12 09:00, bus`	`Parking fee: ₹ 500`	**Pass**
3	Exit time earlier than entry time `Inputs: 2024-05-10 10:00, 2024-05-10 09:00`	`Invalid time`	**Pass**
4	Unknown vehicle type: truck `Inputs: 2024-05-10 09:00, 2024-05-10 10:00, truck`	`Invalid vehicle`	**Pass**
5	Wrong date format: 10-05-2024 `Inputs: 10-05-2024, 11-05-2024`	`Invalid time format`	**Pass**

9. Output



Output screenshot – Program 15 (code and output)

10. Conclusion

The program validates the time input, rejects wrong vehicle types and calculates the fee from the duration and hourly rate with a maximum limit. The test cases for normal, long-stay and invalid inputs gave the expected results.

Program 16: Multi-Day Water Tank Simulation

1. Problem Statement

Process changing inflow and outflow quantities, update water levels, detect overflow and shortages, and determine the first interval in which the tank reaches a specified threshold.

2. Objective

To simulate the water level of a tank day by day using the daily inflow and outflow, detect overflow and shortage, and find the first day on which the target level is reached.

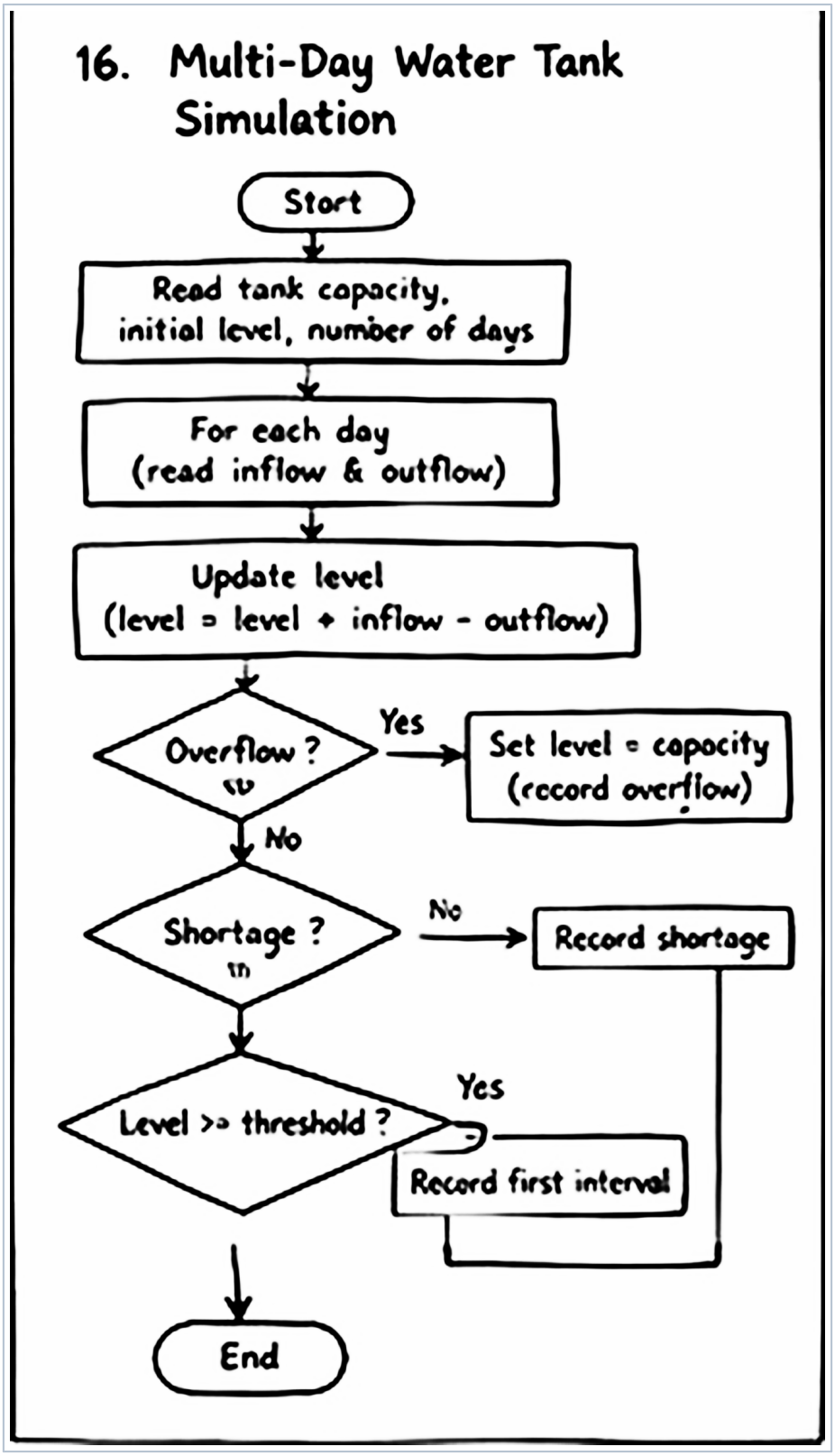
3. Assumptions

- The simulation runs for a maximum of 7 days.
- The initial level, capacity, target level, inflow and outflow are whole numbers entered by the user.
- If the level goes above the capacity, the extra water is overflow and the level is set to the capacity.
- If the level goes below 0, the missing water is shortage and the level is set to 0.
- The simulation stops on the first day when the level is greater than or equal to the target level.

4. Algorithm

1. Start. Read the initial level, tank capacity and target level.
2. For day = 1 to 7: read the inflow and the outflow.
3. Update level = level + inflow – outflow.
4. If level > capacity, display the overflow (level – capacity) and set level = capacity.
5. If level < 0, display the shortage (–level) and set level = 0.
6. Display the water level for the day.
7. If level ≥ target, display the day number as the day the threshold is reached and stop.
8. Otherwise continue with the next day.

5. Flowchart



Flowchart – Program 16: Multi-Day Water Tank Simulation

6. Pseudocode

```
START
  READ level, capacity, threshold
  FOR day ← 1 TO 7 DO
    READ inflow, outflow
    level ← level + inflow - outflow
    IF level > capacity THEN
      DISPLAY "Overflow:", level - capacity
      level ← capacity
    ENDIF
    IF level < 0 THEN
      DISPLAY "Shortage:", -level
      level ← 0
    ENDIF
    DISPLAY "Day", day, "Water:", level
    IF level ≥ threshold THEN
      DISPLAY "Threshold reached on day", day
      EXIT LOOP
    ENDIF
  ENDFOR
END
```

7. Python Source Code

```
level = int(input("Initial water level: "))
capacity = int(input("Tank capacity: "))
threshold = int(input("Target level: "))

for day in range(1, 8):
    inflow = int(input("Inflow: "))
    outflow = int(input("Outflow: "))
    level += inflow - outflow

    if level > capacity:
        print("Overflow:", level - capacity)
        level = capacity
    if level < 0:
        print("Shortage:", -level)
        level = 0

    print("Day", day, "Water:", level)
    if level >= threshold:
        print("Threshold reached on day", day)
        break
```

8. Test Cases

No.	Input	Expected Output	Result
1	Level 20, capacity 100, target 50; day 1: in 40, out 5 `Inputs: 20, 100, 50, 40, 5`	`Day 1 Water: 55` `Threshold reached on day 1`	**Pass**

2	Level 10, capacity 100, target 60; days: 20/5, 30/10, 40/5
	`Inputs: 10, 100, 60, 20, 5, 30, 10, 40, 5`
		`Day 1 Water: 25`
		`Day 2 Water: 45`
		`Day 3 Water: 80`
		`Threshold reached on day 3`
			Pass

3	Level 59, capacity 100, target 89; day 1: in 80, out 15
	`Inputs: 59, 100, 89, 80, 15`
		`Overflow: 24`
		`Day 1 Water: 100`
		`Threshold reached on day 1`
			Pass

4	Level 10, capacity 50, target 40; days: 5/30, 45/0
	`Inputs: 10, 50, 40, 5, 30, 45, 0`
		`Shortage: 15`
		`Day 1 Water: 0`
		`Day 2 Water: 45`
		`Threshold reached on day 2`
			Pass

9. Output

Code

```python
level = int(input("Initial water level: "))
capacity = int(input("Tank capacity: "))
threshold = int(input("Target level: "))

for day in range(1, 8):
    inflow = int(input("Inflow: "))
    outflow = int(input("Outflow: "))
    level += inflow - outflow

    if level > capacity:
        print("Overflow:", level - capacity)
        level = capacity
    if level < 0:
        print("Shortage:", -level)
        level = 0

    print("Day", day, "Water:", level)
    if level >= threshold:
        print("Threshold reached on day", day)
        break
```

Output

```
/python.py/16.py"
Initial water level: 59
Tank capacity: 100
Target level: 89
Inflow: 80
Outflow: 15
Overflow: 24
Day 1 Water: 100
Threshold reached on day 1
```

Output screenshot – Program 16 (code and output)

10. Conclusion

The program updates the tank level every day, handles overflow and shortage correctly, and reports the first day on which the target level is reached. The test cases confirmed all three situations.

Program 17: Student Performance Trend Analysis

1. Problem Statement

Compare assessment scores across multiple examinations, calculate improvements and declines, identify consistently weak subjects, and classify students according to defined criteria.

2. Objective

To compare the previous and current scores of each subject, calculate the improvement or decline, identify the consistently weak subjects, and decide whether the student passes using the average score.

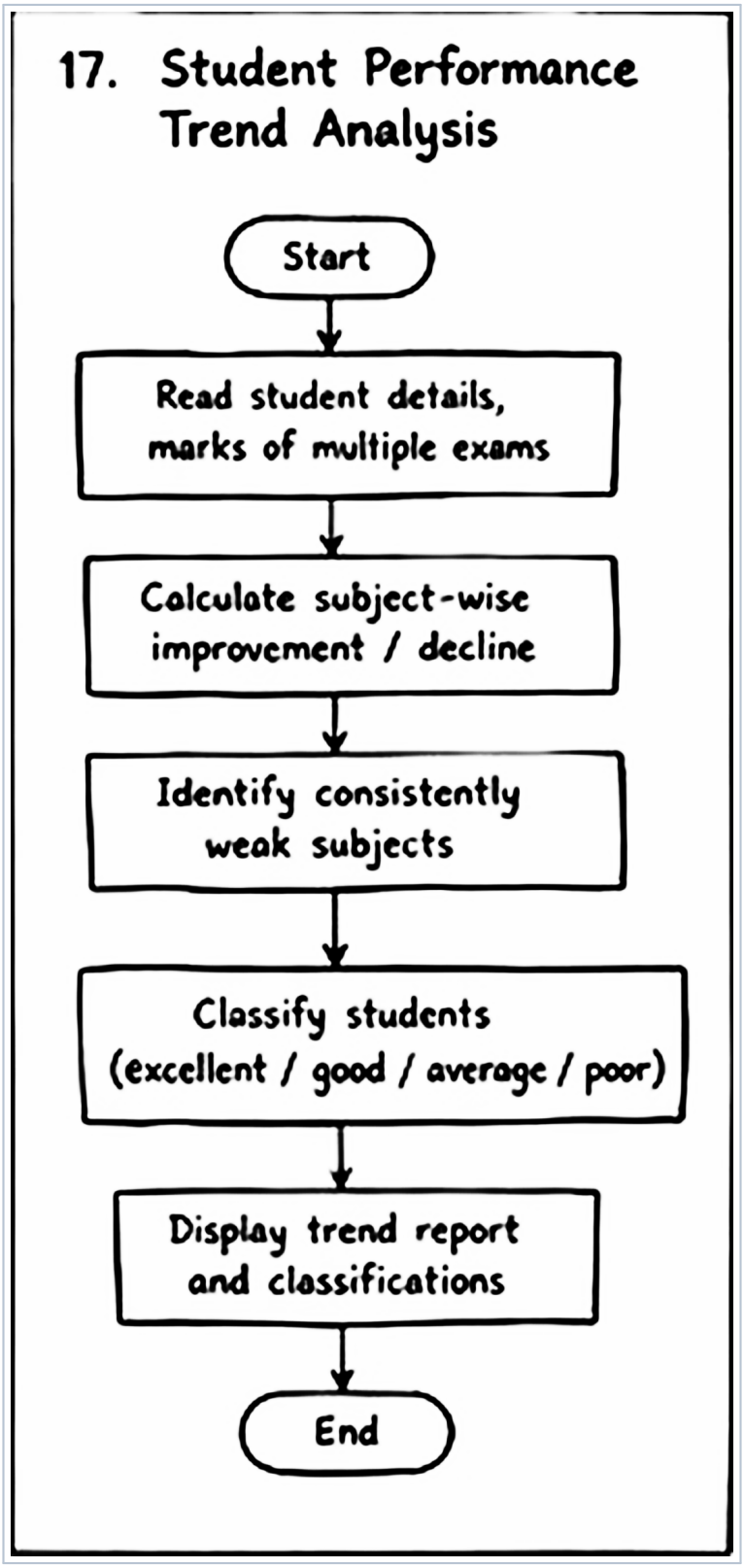
3. Assumptions

- Scores are whole numbers from 0 to 100, and two examinations (previous and current) are compared.
- Change = current score – previous score. A negative value is a decline.
- A subject is consistently weak if the score is below 40 in both examinations.
- The student is classified as Pass if the average of the current scores is 40 or more, otherwise Fail.

4. Algorithm

1. Start. Read the number of subjects.
2. For each subject, read the previous score and the current score and store them in two lists.
3. For each subject, display the change = current – previous.
4. Find the weak subjects: those with previous score < 40 and current score < 40.
5. Display the list of weak subjects.
6. Compute the average of the current scores and display it.
7. If the average ≥ 40 display Pass, otherwise Fail.
8. Stop.

5. Flowchart



Flowchart – Program 17: Student Performance Trend Analysis

6. Pseudocode

```
START
  READ subjects
  old ← empty list, new ← empty list
  FOR i ← 1 TO subjects DO
    READ previous, current
    ADD previous TO old
    ADD current TO new
  ENDFOR
  FOR i ← 1 TO subjects DO
    DISPLAY "Subject", i, "Change:", new[i] - old[i]
  ENDFOR
  weak ← subjects i WHERE old[i] < 40 AND new[i] < 40
  DISPLAY weak
  average ← SUM(new) / subjects
  DISPLAY average
  IF average ≥ 40 THEN
    DISPLAY "Pass"
  ELSE
    DISPLAY "Fail"
  ENDIF
END
```

7. Python Source Code

```
subjects = int(input("Number of subjects: "))
old, new = [], []

for i in range(subjects):
    old.append(int(input("Previous score: ")))
    new.append(int(input("Current score: ")))

for i in range(subjects):
    print("Subject", i + 1, "Change:", new[i] - old[i])

weak = [i + 1 for i in range(subjects) if old[i] < 40 and new[i] < 40]
print("Weak subjects:", weak)

avg = sum(new) / subjects
print("Average:", avg)
print("Result:", "Pass" if avg >= 40 else "Fail")
```

8. Test Cases

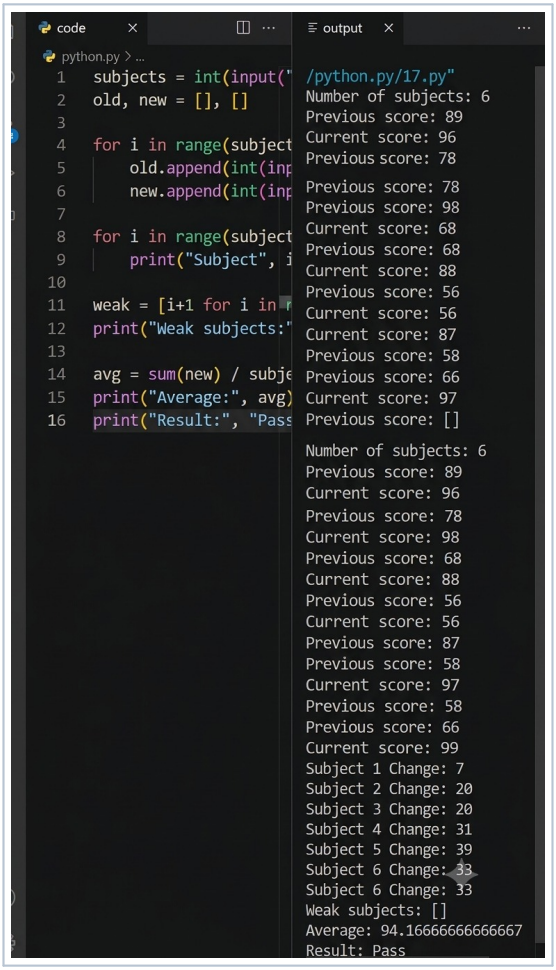
No.	Input	Expected Output	Result

1	3 subjects: (70,80), (35,38), (60,62)
	`Inputs: 3, 70, 80, 35, 38, 60, 62`
		`Subject 1 Change: 10`
		`Subject 2 Change: 3`
		`Subject 3 Change: 2`
		`Weak subjects: [2]`
		`Average: 60.0`
		`Result: Pass`
			Pass

2	2 subjects: (30,32), (35,20)
	`Inputs: 2, 30, 32, 35, 20`
		`Subject 1 Change: 2`
		`Subject 2 Change: -15`
		`Weak subjects: [1, 2]`
		`Average: 26.0`
		`Result: Fail`
			Pass

3	2 subjects: (50,60), (45,55)
	`Inputs: 2, 50, 60, 45, 55`
		`Subject 1 Change: 10`
		`Subject 2 Change: 10`
		`Weak subjects: []`
		`Average: 57.5`
		`Result: Pass`
			Pass

9. Output



Output screenshot – Program 17 (code and output)

10. Conclusion

The program shows the trend of every subject, lists the consistently weak subjects and gives the final result from the average score. The test cases for improvement, decline and weak subjects gave correct results.

Program 18: Fuel Consumption and Route Comparison

1. Problem Statement

Calculate fuel requirements and journey costs for several vehicles using distance, mileage, fuel prices, and reserve requirements; identify the lowest-cost feasible option.

2. Objective

To calculate the fuel needed and the journey cost for each vehicle, and to find the vehicle with the lowest cost.

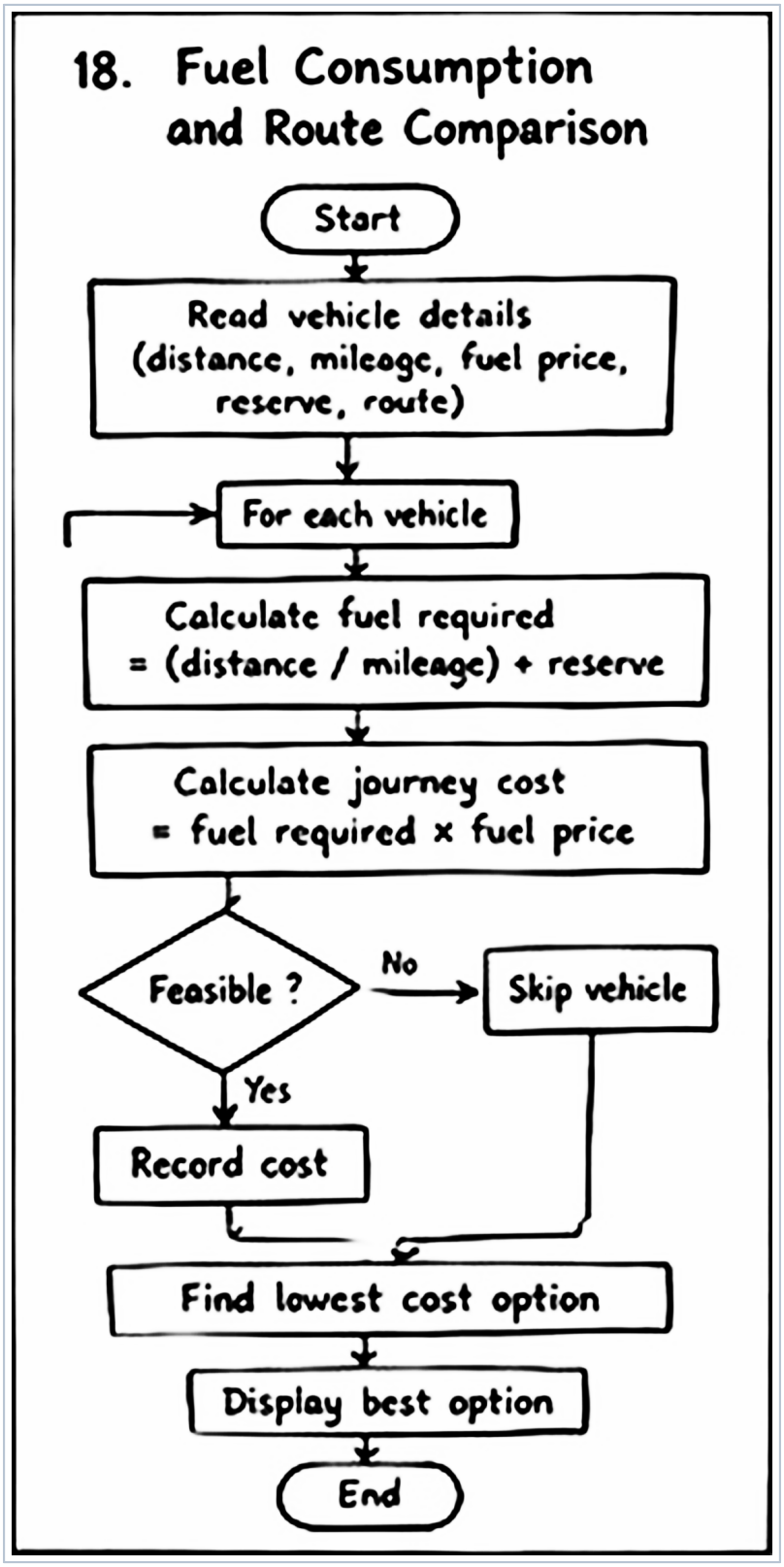
3. Assumptions

- All vehicles travel the same journey distance and use the same fuel price and reserve fuel.
- Fuel required = distance / mileage + reserve fuel (in litres). Journey cost = fuel required × price per litre.
- Mileage is greater than 0, and every vehicle entered is considered feasible for the journey.
- If two vehicles have the same cost, the vehicle entered first is chosen.

4. Algorithm

1. Start. Read the number of vehicles, the journey distance, the fuel price per litre and the reserve fuel.
2. Set best = none.
3. For each vehicle, read the name and the mileage.
4. Compute fuel = distance / mileage + reserve and cost = fuel × price. Display them.
5. If best is none or cost is lower than the best cost, set best = (name, cost).
6. After all vehicles, display the lowest-cost option and its cost.
7. Stop.

5. Flowchart



Flowchart – Program 18: Fuel Consumption and Route Comparison

6. Pseudocode

```
START
  READ n, distance, price, reserve
  best ← NONE
  FOR i ← 1 TO n DO
    READ name, mileage
    fuel ← distance / mileage + reserve
    cost ← fuel × price
    DISPLAY name, fuel, cost
    IF best = NONE OR cost < best.cost THEN
      best ← (name, cost)
    ENDIF
  ENDFOR
  DISPLAY "Lowest-cost option:", best.name
  DISPLAY "Cost:", best.cost
END
```

7. Python Source Code

```
n = int(input("Number of vehicles: "))
distance = float(input("Journey distance (km): "))
price = float(input("Fuel price per litre: "))
reserve = float(input("Reserve fuel (litres): "))

best = None

for i in range(n):
    name = input("Vehicle name: ")
    mileage = float(input("Mileage (km/l): "))
    fuel = distance / mileage + reserve
    cost = fuel * price
    print(name, "Fuel:", round(fuel, 2), "Cost:", round(cost, 2))

    if best is None or cost < best[1]:
        best = (name, cost)

print("Lowest-cost option:", best[0])
print("Cost:", round(best[1], 2))
```

8. Test Cases

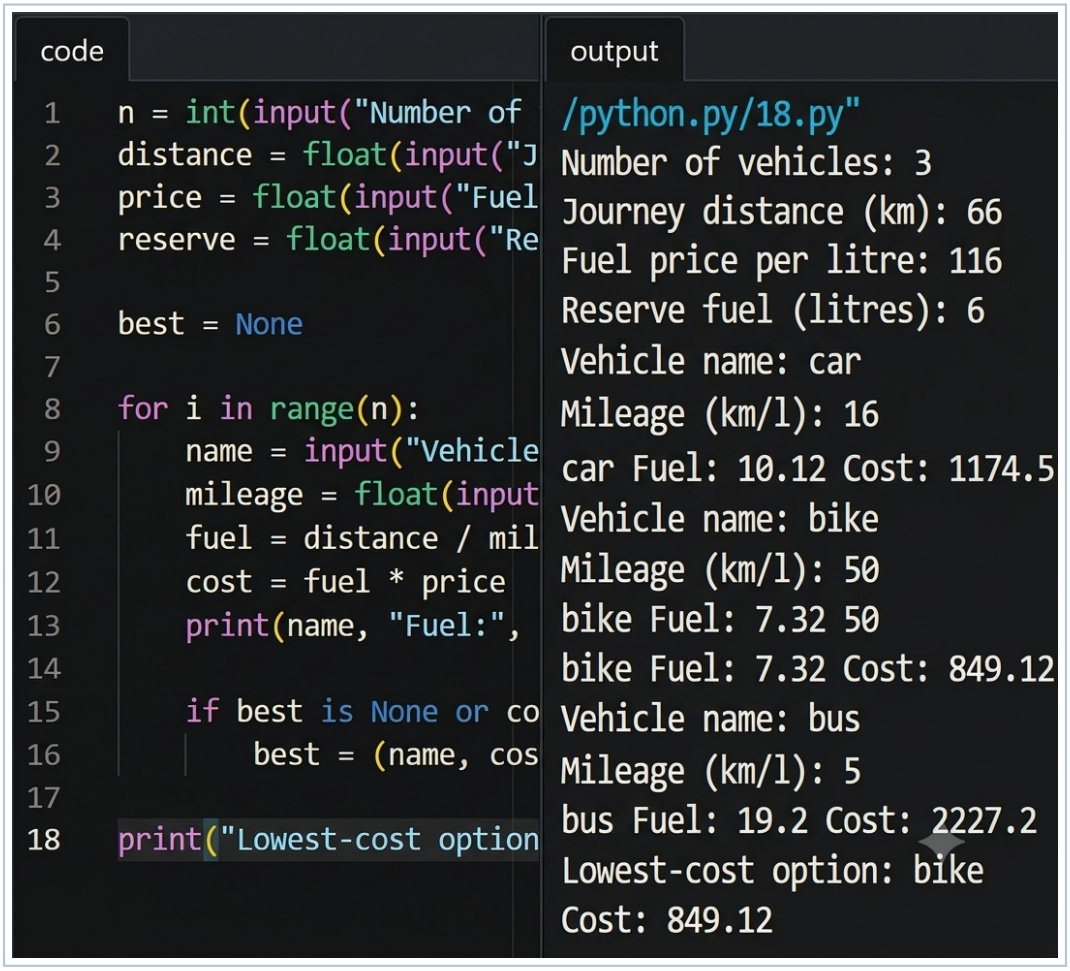
No.	Input	Expected Output	Result

1	2 vehicles, 100 km, Rs 100/l, reserve 5 l: car 20, bike 50
	`Inputs: 2, 100, 100, 5, car, 20, bike, 50`
		`car Fuel: 10.0 Cost: 1000.0`
		`bike Fuel: 7.0 Cost: 700.0`
		`Lowest-cost option: bike`
		`Cost: 700.0`
			Pass

2	1 vehicle, 50 km, Rs 90/l, reserve 2 l: scooter 40
	`Inputs: 1, 50, 90, 2, scooter, 40`
		`scooter Fuel: 3.25 Cost: 292.5`
		`Lowest-cost option: scooter`
		`Cost: 292.5`
			Pass

3	3 vehicles, 66 km, Rs 116/l, reserve 6 l: car 16, bike 50, bus 5
	`Inputs: 3, 66, 116, 6, car, 16, bike, 50, bus, 5`
		`car Fuel: 10.12 Cost: 1174.5`
		`bike Fuel: 7.32 Cost: 849.12`
		`bus Fuel: 19.2 Cost: 2227.2`
		`Lowest-cost option: bike`
		`Cost: 849.12`
			Pass

9. Output



Output screenshot – Program 18 (code and output)

10. Conclusion

The program calculates the fuel and cost of each vehicle and selects the cheapest one. The test cases, including the three-vehicle example, gave the correct lowest-cost option.

Program 19: Multi-Product Shopping Budget Constraint

1. Problem Statement

Evaluate a shopping basket containing multiple products and quantities, apply discount conditions, and determine which purchases must be reduced to remain within a fixed budget.

2. Objective

To calculate the cost of every product with the discount, find the total of the basket, and check whether it is within the budget or by how much the purchases must be reduced.

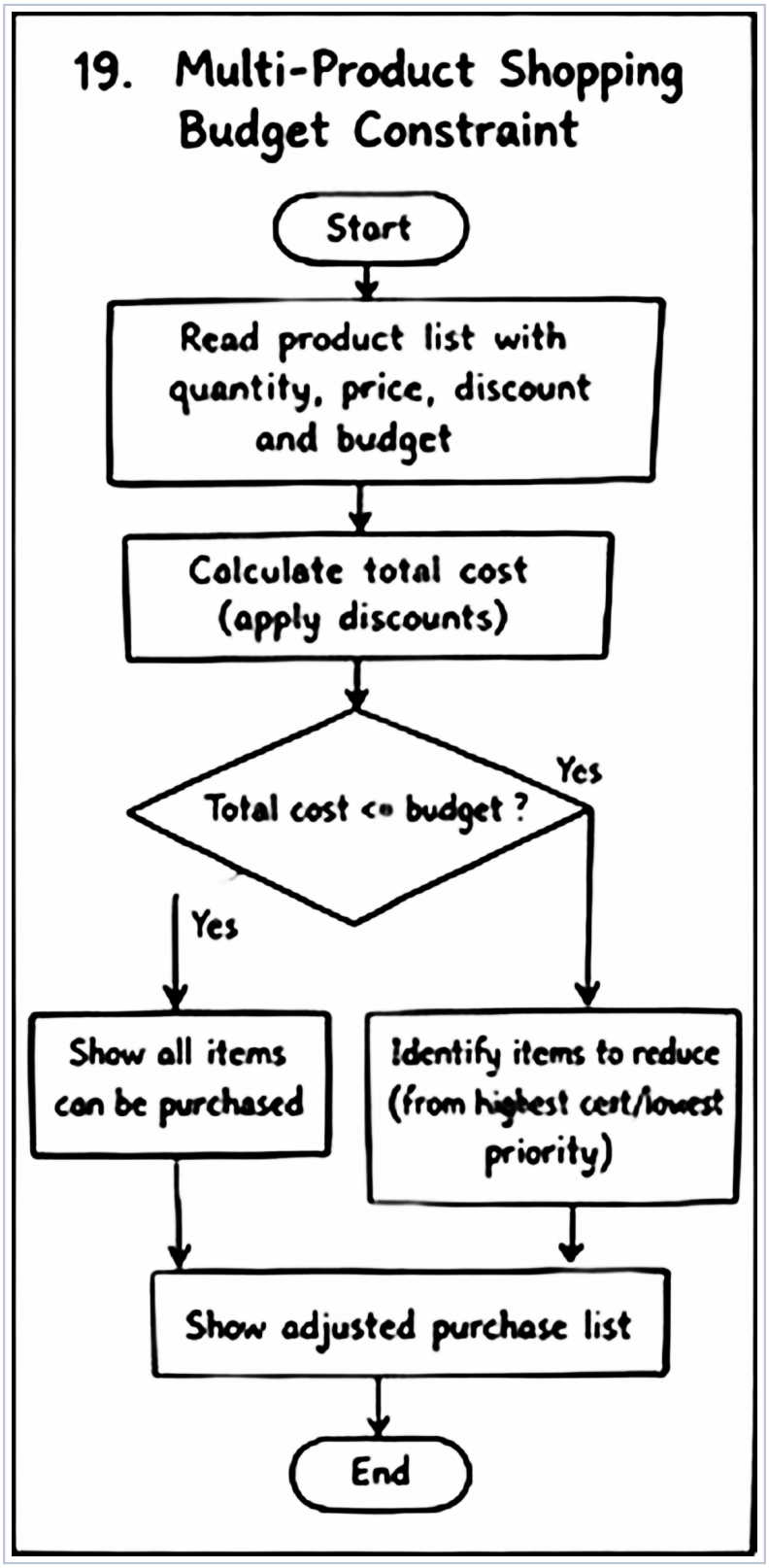
3. Assumptions

- The price is not negative and the quantity is a whole number of 1 or more.
- A 10% discount is given on a product when its quantity is 2 or more.
- Total = sum of the (discounted) product costs, compared with the fixed budget.
- If the total is above the budget, the program shows the amount by which the basket must be reduced; the user then removes items worth at least that amount.

4. Algorithm

1. Start. Read the budget and the number of products n. Set total = 0.
2. For each product, read the name, price and quantity. Compute cost = price × quantity.
3. If the discount condition is true (quantity ≥ 2), multiply cost by 0.90.
4. Add cost to total and display the product cost.
5. Display the total.
6. If total > budget, display the amount to reduce (total – budget); otherwise display "Within budget".
7. Stop.

5. Flowchart



Flowchart – Program 19: Multi-Product Shopping Budget Constraint

6. Pseudocode

```
START
  READ budget, n
  total ← 0
  FOR i ← 1 TO n DO
    READ name, price, qty
    cost ← price × qty
    IF qty > 0 AND price ≥ 0 AND qty ≥ 2 THEN
      cost ← cost × 0.90
    ENDIF
    total ← total + cost
    DISPLAY name, cost
  ENDFOR
  DISPLAY total
  IF total > budget THEN
    DISPLAY "Reduce purchases by:", total - budget
  ELSE
    DISPLAY "Within budget"
  ENDIF
END
```

7. Python Source Code

```
budget = float(input("Enter budget: "))
n = int(input("Number of products: "))
total = 0

for i in range(n):
    name = input("Product name: ")
    price = float(input("Price: "))
    qty = int(input("Quantity: "))
    cost = price * qty

    if qty > 0 and price >= 0 and qty >= 2:
        cost *= 0.90

    total += cost
    print(name, "Cost:", round(cost, 2))

print("Total:", round(total, 2))
if total > budget:
    print("Reduce purchases by:", round(total - budget, 2))
else:
    print("Within budget")
```

8. Test Cases

No.	Input	Expected Output	Result

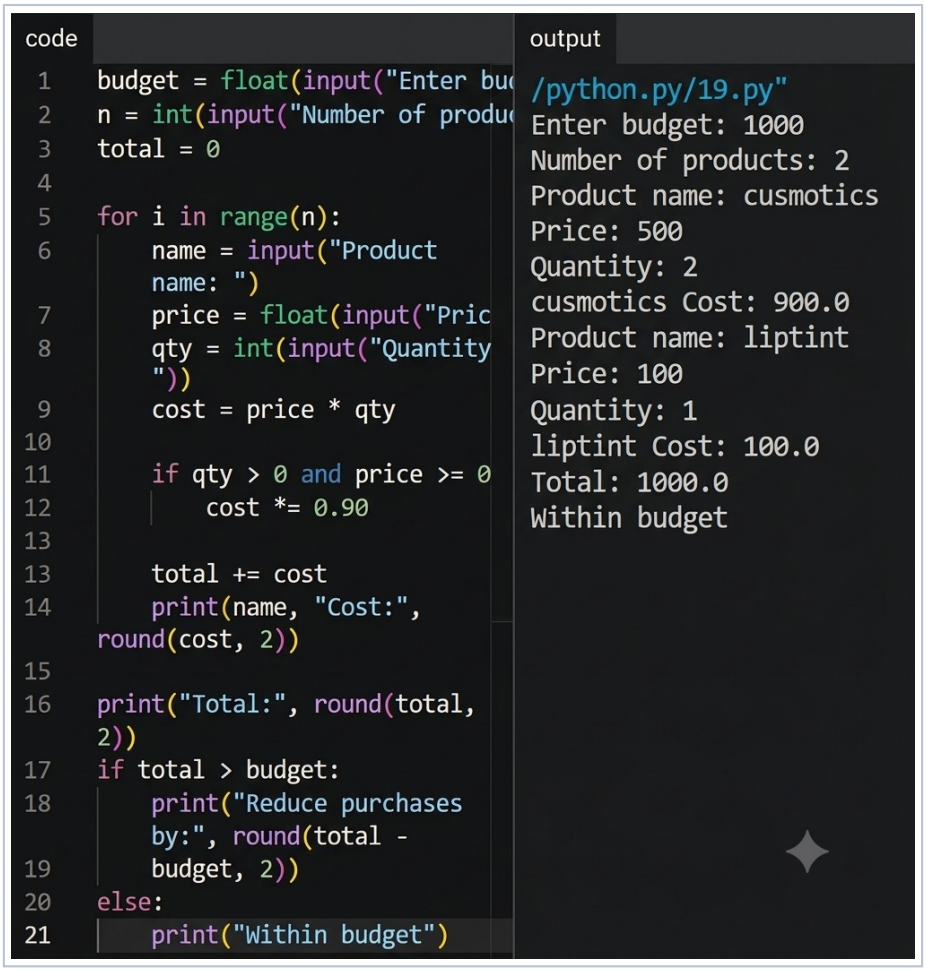
1	Budget 1000: cosmetics 500 x2, lipstick 100 x1
	`Inputs: 1000, 2, cosmetics, 500, 2, lipstick, 100, 1`
		`cosmetics Cost: 900.0`
		`lipstick Cost: 100.0`
		`Total: 1000.0`
		`Within budget`
			Pass

2	Budget 500: pen 20 x5, book 300 x1
	`Inputs: 500, 2, pen, 20, 5, book, 300, 1`
		`pen Cost: 90.0`
		`book Cost: 300.0`
		`Total: 390.0`
		`Within budget`
			Pass

3	Budget 300: shoes 400 x1
	`Inputs: 300, 1, shoes, 400, 1`
		`shoes Cost: 400.0`
		`Total: 400.0`
		`Reduce purchases by: 100.0`
			Pass

4	Budget 1000: bag 600 x2, cap 100 x1
	`Inputs: 1000, 2, bag, 600, 2, cap, 100, 1`
		`bag Cost: 1080.0`
		`cap Cost: 100.0`
		`Total: 1180.0`
		`Reduce purchases by: 180.0`
			Pass

9. Output



Output screenshot – Program 19 (code and output)

10. Conclusion

The program applies the discount rule, finds the basket total and clearly tells whether the budget is met or how much must be reduced. All test cases gave the expected totals.

Program 20: University Admission Decision Engine

1. Problem Statement

Evaluate candidates using subject eligibility, aggregate marks, course-specific thresholds, and tie-breaking criteria; generate a ranked list of eligible applicants.

2. Objective

To check the subject eligibility of every candidate, keep only the eligible candidates, and display them in rank order according to their aggregate marks.

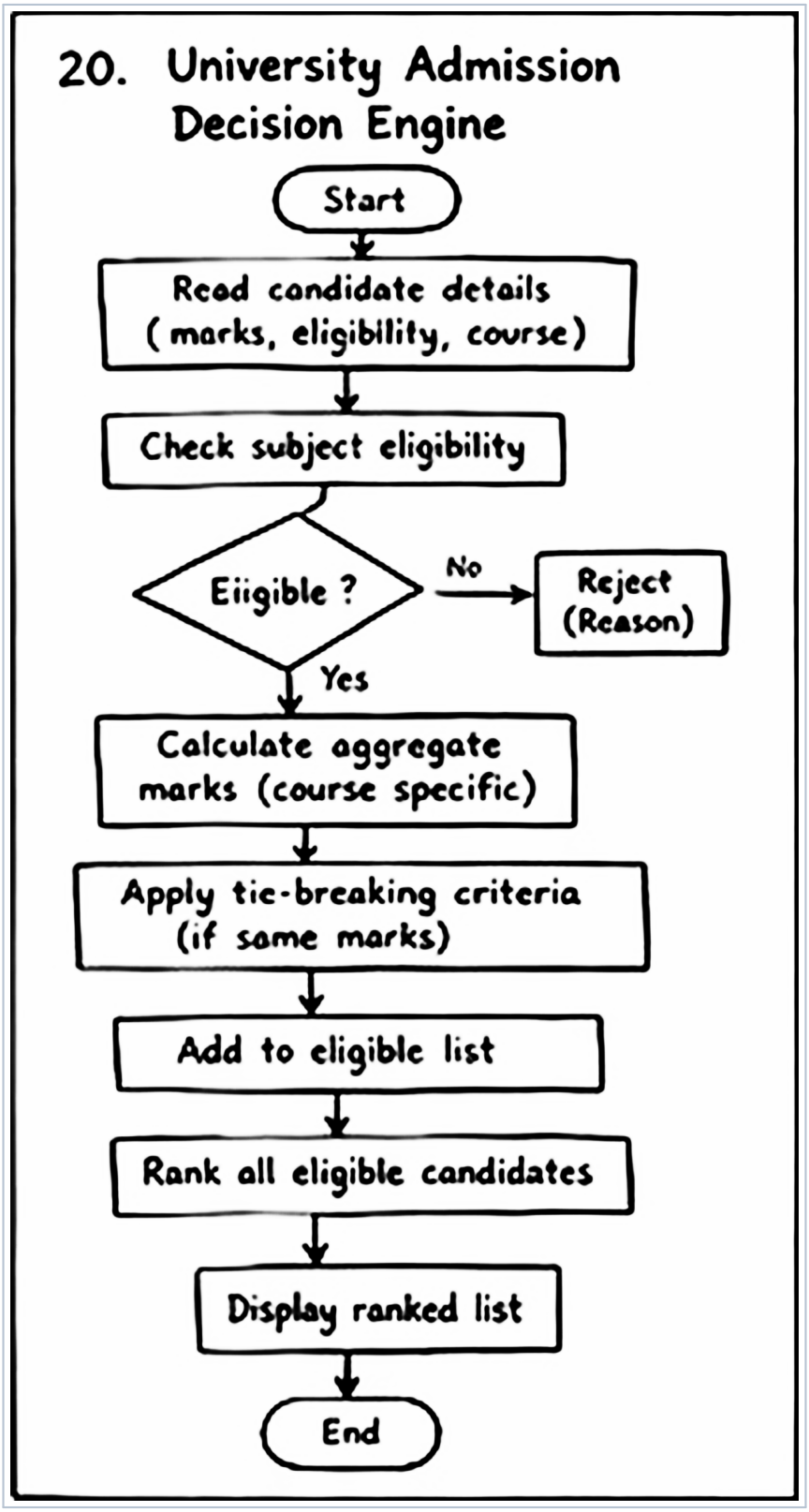
3. Assumptions

- A candidate is eligible only if both Math and Science marks are 35 or more.
- The aggregate marks are entered for each candidate, and a higher aggregate gives a better rank.
- When two candidates have the same aggregate, the candidate entered first gets the better rank (the sort keeps the entry order).
- Candidates who are not eligible are left out of the ranked list. If nobody is eligible, only the heading is shown.

4. Algorithm

1. Start. Read the number of candidates n and create an empty list candidates.
2. For each candidate, read the name, Math marks, Science marks and aggregate marks.
3. If Math ≥ 35 and Science ≥ 35, add (name, aggregate) to the list.
4. Sort the list in descending order of aggregate marks (equal marks keep entry order).
5. Display "Admission Ranking:".
6. Display rank, name and aggregate marks for every candidate in the sorted list.
7. Stop.

5. Flowchart



Flowchart – Program 20: University Admission Decision Engine

6. Pseudocode

```
START
  READ n
  candidates ← empty list
  FOR i ← 1 TO n DO
    READ name, math, science, aggregate
    IF math ≥ 35 AND science ≥ 35 THEN
      ADD (name, aggregate) TO candidates
    ENDIF
  ENDFOR
  SORT candidates BY aggregate (highest first, ties keep entry order)
  DISPLAY "Admission Ranking:"
  rank ← 1
  FOR EACH (name, marks) IN candidates DO
    DISPLAY rank, name, marks
    rank ← rank + 1
  ENDFOR
END
```

7. Python Source Code

```
n = int(input("Number of candidates: "))
candidates = []

for i in range(n):
    name = input("Candidate name: ")
    math = int(input("Math marks: "))
    science = int(input("Science marks: "))
    total = int(input("Aggregate marks: "))

    if math >= 35 and science >= 35:
        candidates.append((name, total))

candidates.sort(key=lambda x: -x[1])

print("Admission Ranking:")
for rank, (name, marks) in enumerate(candidates, 1):
    print(rank, name, marks)
```

8. Test Cases

No.	Input	Expected Output	Result

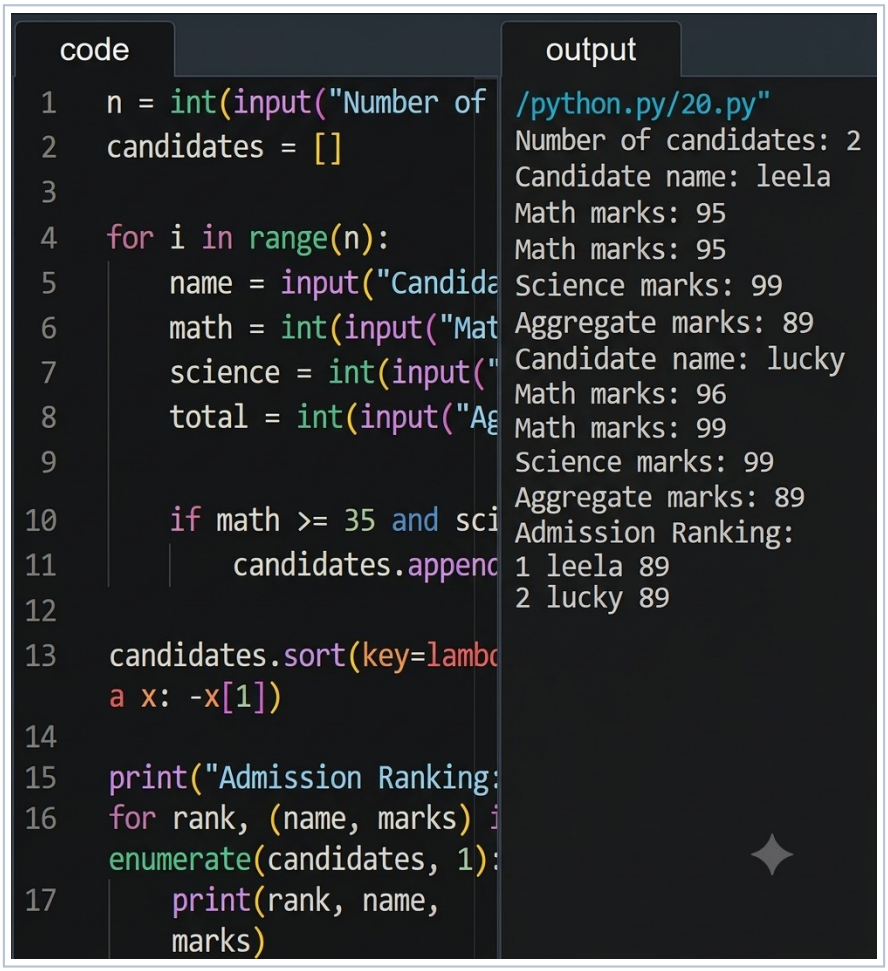
1	3 candidates: Asha(80,90,85), Ravi(30,95,90), Meena(70,60,88)
	`Inputs: 3, Asha, 80, 90, 85, Ravi, 30, 95, 90, Meena, 70, 60, 88`
		`Admission Ranking:`
		`1 Meena 88`
		`2 Asha 85`
			Pass

2	2 candidates with a tie: Leela(95,99,89), Lucky(99,99,89)
	`Inputs: 2, Leela, 95, 99, 89, Lucky, 99, 99, 89`
		`Admission Ranking:`
		`1 Leela 89`
		`2 Lucky 89`
			Pass

3	2 candidates, none eligible `Inputs: 2, A, 20, 30, 70, B, 10, 20, 60`	`Admission Ranking:`	**Pass**

4	3 candidates: Kiran(60,70,75), Divya(65,80,92), Sai(90,85,92)
	`Inputs: 3, Kiran, 60, 70, 75, Divya, 65, 80, 92, Sai, 90, 85, 92`
		`Admission Ranking:`
		`1 Divya 92`
		`2 Sai 92`
		`3 Kiran 75`
			Pass

9. Output



Output screenshot – Program 20 (code and output)

10. Conclusion

The program filters out ineligible candidates and produces a correct ranked list, with ties resolved by order of entry. The test cases for normal, tie and no-eligible-candidate situations gave correct results.